

company-management — idei de endpoint-uri

Ce mai poți construi pe modelul pe care îl ai deja · Rareș · revizuit 3 septembrie 2026

Înainte de orice de aici: închide întâi  -urile din `CODE_REVIEW.md` runda 5. Fiecare endpoint nou moștenește problemele celor vechi — mai ales lipsa lui `@RestControllerAdvice`, care face ca orice endpoint nou să răspundă 500 în loc de 404.

Reguli care se aplică la toate cele de mai jos:

- id inexistent → **404**, nu 500. Deci: excepție proprie + handler în advice.
- creare reușită → **201 Created** + header `Location`. Ștergere → **204 No Content** sau 200 cu corpul șters.
- colecție goală → **200 []**, niciodată excepție.
- orice câmp care intră din exterior are validare pe DTO, inclusiv `@PathVariable` și `@RequestParam`.
- un `@Modifying` bulk **nu** trece prin Bean Validation — validează tu, înainte.

Două reguli care se aplică la fiecare endpoint din fișa asta:

- Numeri interogărilor, nu te uiți doar la răspuns.** Pornește cu `spring.jpa.show-sql=true`, dă cererea, numără liniile Hibernate: din consolă. Un răspuns corect emis cu 6 interogări în loc de 1 arată identic în Postman și diferit în producție. Fiecare endpoint de mai jos are numărul așteptat scris în „gata când”.
- Scrit tu JPQL-ul → `join fetch`. Nu-l scrii tu (`findById`, `findAll`, interogări derivate), sau returnezi `Page` → `@EntityGraph(attributePaths = ...)`.** Folosește `attributePaths`, nu grafuri numite: un typo în `attributePaths` oprește pornirea, un nume greșit de graf e ignorat în tăcere și rămâi cu N+1 crezând că l-ai rezolvat.

Nivelul 1 — pe fix ce știi deja

derived queries + ce ai scris până acum

1. `GET /api/departments/{id}` — un singur departament

ușor

Ai listarea tuturor, dar n-ai cum să ceri unul singur. E endpoint-ul care lipsește ca să poți da `Location` după un POST.

```
GET /api/departments/3
{"id":3,"name":"Finance","location":"Iasi","budget":180000.0,"employees":[...]}
```

Gata când: id existent → 200 cu departamentul · id inexistent → 404 cu mesaj, nu 500 · **exact 1 linie**
Hibernate: în consolă.

Scris în `e642a5f`, criteriul nu e atins: emite 2 interogări — `findById`, apoi încă una când `DepartmentResponse.from` atinge colecția `LAZY`. Nu poți pune `join fetch` pe `findById`, pentru că nu tu i-ai scris JPQL-ul. Redecar-o în interfața ta și pune-i un graf deasupra: `@EntityGraph(attributePaths = "employees")`.

2. GET /api/employees/{id} — un singur angajat

ușor

Perechea celui de sus, pe cealaltă entitate. Îl vei folosi în teste ca să verifici ce a scris un PUT sau un PATCH — chiar acum n-ai cum să citești salariul unui angajat.

Gata când: răspunsul conține salariul și numele departamentului · 404 pe id inexistent · **exact 1 linie**
Hibernate: .

Scris în e642a5f , criteriul nu e atins: 3 interogări, pentru că EmployeeResponse.from cheamă getDepartment().getName() pe o asociere LAZY . Același tratament: @EntityGraph(attributePaths = "department") pe findById în EmployeeRepository . Observă că graful merge la fel de bine spre un @ManyToOne , nu doar spre colecții.

3. GET /api/departments/{id}/employees — angajații unui departament

ușor

Aceeași informație pe care o ai deja în listarea de departamente, dar adresabilă separat — clientul nu mai trebuie să descarce toate departamentele ca să vadă echipa unuia.

Gata când: departament fără angajați → 200 [] · departament inexistent → 404 (atenție: cele două cazuri sunt diferite, nu le confunda) · **maximum 2 interogări**, una pentru existența departamentului și una pentru angajați — sau 1 singură, cu graf.

Distincția din DoD e tot subiectul: „colecție goală” ≠ „resursa părinte nu există”. Odată ce ai @EntityGraph pe findById de la endpoint-ul 1, asta se rezolvă cu o singură chemare.

4. GET /api/employees?jobTitle=Java Developer — filtrare după funcție

ușor

Primul tău parametru de query. Când lipsește, endpoint-ul se comportă exact ca acum (întoarce tot).

```
GET /api/employees?jobTitle=QA Engineer
[{"id":3,"firstName":"Maria","lastName":"Ionescu","departmentName":"IT"}]
```

Gata când: fără parametru → toți angajații · cu parametru → doar cei care se potrivesc · valoare fără rezultate → 200 [] .

Spring Data îți construiește interogarea din numele metodei: findByJobTitle(String jobTitle) . Nu scrii SQL. @RequestParam(required = false) .

Nivelul 2 — noțiuni noi, dar vecine

interogări derivate compuse + relația în ambele sensuri

5. GET /api/employees?minSalary=7000&maxSalary=10000 — interval de salariu

mai greu

Doi parametri opționali care lucrează împreună. Cazul interesant e când vine doar unul dintre ei.

Gata când: ambii → interval închis · doar minSalary → toți peste · doar maxSalary → toți sub · niciunul → toți · minSalary > maxSalary → 400 cu mesaj, nu listă goală.

findBySalaryBetween , findBySalaryGreaterThanEqual , findBySalaryLessThanEqual . Ultima condiție din DoD nu se rezolvă cu o adnotare — e o verificare pe care o scrii tu.

6. PATCH /api/employees/{id}/department — mută angajatul în alt departament

mai greu

Primul endpoint care atinge **relația**, nu doar câmpurile. Corpul cererii: `{"departmentId": 2}`.

```
PATCH /api/employees/1/departament {"departmentId":2}
{"id":1,"firstName":"Rares","lastName":"Dumitru","departmentName":"HR"}
```

Gata când: angajatul apare în lista noului departament **și** a dispărut din a vechiului, verificat prin `GET /api/departments/{id}/employees` pe amândouă · departament inexistent → 404, iar angajatul rămâne unde era.

Ai deja `addEmployee` și `removeEmployee` pe `Department`. Sunt exact uneltele pentru asta — folosește-le pe amândouă, în ordinea corectă. Întrebarea de pus: care capăt al relației e cel pe care baza de date îl citește când scrie `department_id`?

7. GET /api/departments/{id}/payroll — cât costă lunar un departament

mai greu

Prima ta agregare. Suma salariilor angajaților dintr-un departament, calculată **în baza de date**, nu în Java.

```
GET /api/departments/1/payroll
{"departmentId":1,"departmentName":"IT","employeeCount":3,"totalSalary":23800.00}
```

Gata când: rezultatul vine dintr-un singur `SELECT` (verifici cu `show-sql`) · departament fără angajați → `employeeCount: 0` și `totalSalary: 0`, nu `null` · departament inexistent → 404.

`@Query("select sum(e.salary) from Employee e where e.department.id = :id")`. Cazul „fără angajați” e capcana: în SQL, `sum` pe zero rânduri dă `NULL`, nu `0`.

8. GET /api/employees/search?q=pop — căutare după nume, parțială

mai greu

Caută în prenume **sau** nume, fără să conteze literele mari. „pop” găsește și „Popescu”, și „Popa”.

Gata când: „POP”, „pop” și „Pop” dau același rezultat · `q` gol sau lipsă → 400 cu mesaj (o căutare fără termen nu e o căutare) · niciun rezultat → 200 `[]`.

`findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCase(...)`. Da, e lung — asta e semnalul că ai ajuns la limita interogărilor derivate și că un `@Query` ar fi mai lizibil. Scrie-le pe amândouă și compară.

9. GET /api/employees?page=0&size=5&sort=salary,desc — paginare și sortare

provocare

Astăzi, dacă ai 10.000 de angajați, GET /api/employees îi trimite pe toți. Paginarea e răspunsul, și Spring Data ți-o dă aproape gratis.

```
GET /api/employees?page=0&size=2&sort=salary,desc
{"content":[{"...},{...}], "totalElements":7, "totalPages":4, "number":0, "size":2}
```

Gata când: repository-ul primește Pageable și întoarce Page<Employee> · nu construiești tu page / size din @RequestParam · sort=salary,desc chiar sortează · pagină peste ultima → 200 cu content: [] · **totalElements egal cu numărul real de rânduri** — verifică-l explicit, nu-l crede.

Semnătura e findAll(Pageable pageable), iar în controller pui direct Pageable pageable — Spring îl construiește din query string. Page<Employee> conține entități, tu trimiți DTO-uri: caută Page.map.

Capcana de pe ultima condiție: dacă încarci asocierea cu @Query("... join fetch ...") + Page, Spring derivă interogarea de count din JPQL-ul tău, cu tot cu join — și numeri rânduri de join, nu entități. Măsurat pe 4 departamente: totalElements raportat 8. Cu @EntityGraph, count-ul rămâne curat: 4. De-aia regula din caseta de sus spune „returnezi Page → @EntityGraph”.

10. GET /api/departments/stats — un rând pe departament, cu agregări

provocare

Nume, număr de angajați, salariu mediu, salariu maxim — pentru toate departamentele, într-o singură interogare.

```
GET /api/departments/stats
[{"name":"IT","employeeCount":3,"averageSalary":7933.33,"maxSalary":9500.00},
 {"name":"HR","employeeCount":2,"averageSalary":6600.00,"maxSalary":7200.00}]
```

Gata când: un singur SELECT pentru tot răspunsul, verificat prin numărarea liniilor Hibernate: · departamentele fără angajați apar și ele, cu 0 · nu încarci nicio entitate Employee în memorie · **fără buclă peste departamente în serviciu.**

Ai o variantă a acestui endpoint în 3b08cf2, cu răspuns corect și 6 interogări: findAll(), apoi câte o interogare pentru fiecare departament, într-o buclă. Ăsta e N+1-ul clasic — invizibil pe datele din seeder, letal pe 400 de departamente. Un singur group by îl înlocuiește: baza grupează o dată, pentru toate departamentele deodată.

Două drumuri: proiecție pe interfață (declari o interfață cu getName(), getEmployeeCount() ... și Spring o umple singur) sau select new ...DepartmentStats(d.name, count(e), avg(e.salary)). Reține perechea: left join (nu inner) ca departamentele goale să rămână în listă, și count(e.id) (nu count(*)) ca ele să dea 0, nu 1 — cu left join, un departament fără angajați produce un rând cu e null, iar count(*) îl numără.

11. POST /api/departments/{id}/employees/bulk — angajări în lot, tot sau nimic

provocare

Primești o listă de angajați și îi creezi pe toți într-un departament. Dacă **unul singur** e invalid sau are un email duplicat, nu se salvează **niciunul**.

```
POST /api/departments/2/employees/bulk
[ {...ok...}, {...email duplicat...} ]
→ 409, iar GET /api/departments/2/employees arata exact ce era inainte
```

Gata când: lot valid → 201 cu toți angajații creați · lot cu o singură eroare → 4xx **și zero rânduri noi în baza de date**, verificat prin GET după · validarea prinde elementele listei, nu doar lista.

Ultima condiție e cea care te trimite la ceva ce ai mai văzut o dată: `@Valid` pe un `@RequestBody List<...>` nu coboară singur în elemente. Iar „tot sau nimic” e definiția unei tranzacții — ai deja `@Transactional` pe servicii; întrebarea e ce anume declanșează rollback-ul.

12. GET /api/departments/{id}/employees/top?limit=3 — cei mai bine plătiți

provocare

Primii N angajați dintr-un departament, ordonați descrescător după salariu. Limita vine din URL și trebuie să fie rezonabilă.

Gata când: limitarea se face **în baza de date** (SQL-ul conține `limit / fetch first`), nu cu `.stream().limit()` în Java · `limit` lipsă → 3 implicit · `limit=0` sau negativ → 400 · departament cu 2 angajați și `limit=10` → 2 rezultate, nu eroare.

Prima condiție e miezul: dacă filtrezi în Java, ai adus deja toate rândurile din baza de date — exact lucrul pe care încerci să-l eviți. `PageRequest.of(0, limit, Sort.by("salary").descending())`. Numără liniile **Hibernate**: ca să confirmi: una.

Ia-le pe rând, de sus în jos. Câte un commit per endpoint, cu buildul rulat înainte, `grep` după fiecare regulă închisă — și numărate liniile **Hibernate**: înainte să declari endpoint-ul gata.